

Python Security Automation — Log Analyzer per tentativi di login falliti

Linguaggio: Python 3

Ambiente: Kali Linux (VM UTM, Apple Silicon)

Tipo di lab: Blue team / automazione

Obiettivo:

Scrivere da zero, in Python, uno script che replichi (in piccolo) il tipo di analisi che un SOC analyst farebbe manualmente o tramite un SIEM: leggere un log di autenticazione SSH, individuare i tentativi di login falliti, raggrupparli per indirizzo IP, e segnalare automaticamente quali IP superano una soglia sospetta (possibile brute-force/credential stuffing).

Dataset utilizzato:

File di log di esempio in formato SSH auth.log standard, con un pattern intenzionalmente realistico: un IP (203.0.113.45) che tenta l'accesso con più utenti diversi (admin, root, test, postgres) a ripetizione — tipico di un attacco automatizzato — e un secondo IP (198.51.100.7) con un singolo tentativo isolato, oltre a login legittimi riusciti.

Metodologia — costruzione incrementale dello script

1. Verifica ambiente

Problema iniziale: prima di scrivere qualsiasi script, verificato che Python fosse disponibile sull'ambiente.

Comando:

```
python3 --version
```

Esito: inizialmente command not found — Python3 non era installato su questa installazione Kali. Risolto con:

```
sudo apt install -y python3
```

(nota: durante l'installazione si è verificato un errore di battitura — phyton3 invece di python3 — causa comune di falsi "pacchetto non trovato" quando in realtà il problema è solo ortografico)

2. Test di base — lettura del file

Script iniziale (log_analyzer.py):

```
with open("sample_auth.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Esecuzione:

```
python3 log_analyzer.py
```

Esito: tutte le righe del file di log correttamente ristampate a schermo — confermato che Python legge il file senza problemi, prima di aggiungere logica.

3. Riconoscimento e conteggio dei tentativi falliti

Ragionamento: individuato il pattern testuale delle righe di interesse — contengono sempre la parola "Failed", e l'indirizzo IP compare sempre subito dopo la parola "from".

Script aggiornato:

```
from collections import Counter

ip_counter = Counter()

with open("sample_auth.txt", "r") as f:
    for line in f:
        if "Failed" in line:
            parts = line.split()
            index = parts.index("from")
            ip = parts[index + 1]
            ip_counter[ip] += 1
```

```
for ip, count in ip_counter.most_common():
    print(f"{ip}: {count} tentativi falliti")
```

Esecuzione e output:

```
203.0.113.45: 9 tentativi falliti
198.51.100.7: 2 tentativi falliti
```

Risultato coerente con il dataset: l'IP con pattern di attacco su più utenti diversi risulta correttamente il più frequente.

4. Aggiunta di una soglia di allarme configurabile

Ragionamento: un semplice conteggio richiede comunque interpretazione manuale. Aggiunta una soglia (THRESHOLD) per classificare automaticamente ogni IP come "sospetto" o "normale".

Script finale:

```
from collections import Counter

THRESHOLD = 5

ip_counter = Counter()

with open("sample_auth.txt", "r") as f:
    for line in f:
        if "Failed" in line:
            parts = line.split()
            index = parts.index("from")
            ip = parts[index + 1]
            ip_counter[ip] += 1

print("=== Report tentativi di login falliti ===\n")

for ip, count in ip_counter.most_common():
    status = "SOSPETTO" if count >= THRESHOLD else "normale"
    print(f"{ip}: {count} tentativi falliti -> {status}")
```

Esecuzione e output finale:

```
=== Report tentativi di login falliti ===
```

```
203.0.113.45: 9 tentativi falliti -> SOSPETTO
```

```
198.51.100.7: 2 tentativi falliti -> normale
```

Conclusioni:

Lo script, pur semplice, replica concretamente la logica di base di una detection rule "brute-force" come quelle che si troverebbero in un SIEM reale (Wazuh, Splunk, Sentinel): filtro su pattern testuale, aggregazione per chiave (IP), confronto con soglia. Comprendere questa logica scrivendola da zero rende molto più chiaro cosa succede "sotto il cofano" quando si configurano regole di detection in strumenti professionali.

Cosa ho imparato:

- Uso del modulo `collections.Counter` per aggregazioni/conteggi, alternativa più pulita a un dizionario gestito manualmente
- Parsing di testo non strutturato tramite `split()` e ricerca di posizione (`index()`) invece di regex, quando il formato è sufficientemente prevedibile
- L'indentazione in Python è sintassi, non stile: a differenza di altri linguaggi (Java, C), va scritta esplicitamente e determina l'appartenenza di una riga a un blocco
- Uso di `cat -A` per rendere visibili spazi/tab nascosti quando l'indentazione dà errori
- Tecnica dell'heredoc (`cat > file << 'EOF' ... EOF`) come alternativa a nano per scrivere file da terminale preservando l'indentazione esatta